AEGIS-AGENT

OpenRouter API + Database Integration Guide

Antigravity implementation specification | Repository reviewed: 30 October 2026

Goal

Connect the existing AEGIS-AGENT project to a persistent PostgreSQL database and the team's already-created OpenRouter API key. Implement the integration in the existing project, preserve the current UI and agent workflow, and replace demo-only behavior incrementally with real backend services.

Repository baseline observed

The public main branch README calls the project a front-end prototype and states that real scans and code changes are disabled. package.json describes a frontend prototype and contains only start/dev scripts, with no declared npm dependencies. server.js uses native Node HTTP and imports ./backend/routes. The visible root listing includes css/, js/, README.md, index.html, package.json and server.js; verify whether backend/routes actually exists in the checked-out branch before editing.

Important: Never paste the OpenRouter key into the PDF, source code, GitHub, frontend JavaScript, browser storage, screenshots or chat. Antigravity should create a local .env.example with placeholders and instruct the developers to populate their own ignored .env file.

1. Repository-aware starting audit

Before coding, inspect the full checked-out repository and report the actual directory tree. Do not assume the project is React/Vite/Express: the current package.json identifies a minimal Node server, and server.js uses Node's built-in http module.

• Inspect package.json, server.js, README.md, index.html, css/, js/ and all backend folders.

• Check whether ./backend/routes exists and whether it exports handleApiRequest(req,res). If missing, explain the mismatch and implement a deliberate backend structure.

• Identify current API endpoints and simulation state. Preserve existing endpoint behavior where feasible or document compatibility changes.

• Do not replace the existing frontend or redesign it as part of this task. Focus on database, OpenRouter and backend integration.

2. Target architecture

Browser UI → Node.js backend/API → agent orchestration/service layer → OpenRouter chat completions endpoint. The backend also reads/writes PostgreSQL. Browser never calls OpenRouter directly.

```
Frontend (existing HTML/CSS/JS)
↓ same-origin /api/* requests
Node.js HTTP server / API router
■■■ Run coordinator and agent services
■■■ OpenRouter client (server-only secret)
■■■ PostgreSQL data-access layer
↓
PostgreSQL database
```

Use the existing Node runtime initially. Add Express only if the team deliberately chooses to migrate and verifies all existing routes. Avoid adding a separate Python service unless an actual dependency requires it.

3. OpenRouter API integration

The team has already generated an OpenRouter API key. The developers must place it locally in .env; Antigravity must not ask for the key value or commit it.

OpenRouter endpoint: https://openrouter.ai/api/v1/chat/completions

Base URL: https://openrouter.ai/api/v1

Authentication: Authorization: Bearer $OPENROUTER_API_KEY

Model: read from environment configuration; do not hardcode a model name. Confirm the selected model is available on OpenRouter and check its tool/JSON support before relying on those capabilities.

Create a single reusable backend module, e.g. backend/services/llmClient.js, that accepts model, system prompt, user content, temperature and token budget; handles timeout, provider errors, rate limits, response parsing and safe logging. Agents call this module with distinct prompts and bounded inputs.

Example environment file:

```
PORT=3000
DATABASE_URL=postgresql://USER:PASSWORD@HOST:5432/postgres
OPENROUTER_API_KEY=replace_with_your_local_key
OPENROUTER_BASE_URL=https://openrouter.ai/api/v1
OPENROUTER_MODEL=replace_with_a_model_id
APP_ENV=development
```

Example backend request (illustrative):

```
const response = await fetch(`${process.env.OPENROUTER_BASE_URL}/chat/completions`, {
method: "POST",
headers: {
"Authorization": `Bearer ${process.env.OPENROUTER_API_KEY}`,
"Content-Type": "application/json",
"HTTP-Referer": process.env.APP_URL || "http://localhost:3000",
"X-Title": "AEGIS-AGENT"
},
body: JSON.stringify({
```

```
model: process.env.OPENROUTER_MODEL,
messages: [{ role: "system", content: systemPrompt },
{ role: "user", content: userPrompt }],
temperature: 0.2
})
});
```

Use the current official OpenRouter API documentation for exact request options and provider-specific capabilities. Do not assume every model supports identical parameters.

4. Agent integration rules

Agent	What to send to LLM	What must remain deterministic/tool-backed
Red Agent	Summarize attack surface and help prioritize scanner results.	Actual probing must come from an approved scanner/tool; no free-form public target attacks.
Sentinel Agent	Assess evidence, explain likely false positives and produce a review rationale.	Evidence collection, reproduction and final verification status must be based on actual outputs.
Fix Agent	Analyze minimal source context and finding evidence; propose a patch and explanation.	Do not execute generated commands or apply patches automatically. Human approval required.
Auditor Agent	Explain test output and compare before/after results.	Run allowlisted tests; test pass/fail is determined by process exit status and test evidence, not LLM opinion.

Do not call the LLM separately from the browser for each agent. All model requests must pass through the backend client. Start with one configured model for all agents; support per-agent model overrides through configuration only after the shared integration works.

5. PostgreSQL database setup

Recommended initial option: PostgreSQL. For a hosted hackathon setup, Supabase PostgreSQL is convenient; local PostgreSQL is also acceptable. This is a database choice, not an AI API key. The backend connects using DATABASE_URL and a database password.

Setup steps for the team:

• Create a PostgreSQL database (Supabase project or local PostgreSQL instance).

• Copy the PostgreSQL connection string from the provider dashboard; use the server-side connection string, not a browser/public client key.

• Set a strong database password and keep it private. If using Supabase, use the direct or session-pooler connection option appropriate for the Node runtime and deployment.

• Add DATABASE_URL to local .env; ensure .env is ignored by Git.

• Create versioned SQL migrations and run them from the backend project.

• Use a least-privilege database user for the application where practical; do not use an owner/admin credential in production.

6. Initial database schema

Table	Purpose / key columns
users	id, email, display_name, password_hash or external_auth_id, created_at. Only add if authentication is actually implemented.
scan_runs	id, target_label, scope_id, status, started_at, completed_at, run_mode, created_by.
findings	id, run_id, title, description, severity, cwe, source_tool, asset, file_path, line_start, line_end, verification_status, fingerprint, created_at.
evidence	id, finding_id, evidence_type, sanitized_content, storage_ref, collected_at. Avoid storing raw secrets.
agent_events	id, run_id, agent_name, stage, status, summary, input_ref, output_ref, started_at, completed_at, duration_ms.

Table	Purpose / key columns
remediation_proposals	id, finding_id, model_id, rationale, diff_text, status, created_at, reviewed_by, reviewed_at.
approval_records	id, proposal_id, decision, reviewer, reason, decided_at.
regression_results	id, finding_id, proposal_id, test_name, test_type, status, output_summary, executed_at.
model_usage	id, run_id, agent_name, provider, model_id, prompt_tokens, completion_tokens, cost_if_returned, created_at.

Add foreign keys, indexes (run_id, finding_id, status, created_at), timestamps and appropriate delete behavior. Store large scanner logs or patch files in bounded text/object storage when needed; keep database rows as metadata and references. Do not store API keys, passwords or unredacted secrets in these tables.

7. Backend routes to implement

Method + route	Purpose
GET /api/health	Health check; do not expose secrets or connection strings.
GET /api/runs	List recent runs from database.
POST /api/runs	Create a run record and initiate the existing safe workflow.
GET /api/runs/:id	Read run status, findings and agent timeline.
GET /api/findings/:id	Read finding details and sanitized evidence.
POST /api/findings/:id/propose-fix	Call Fix Agent via backend OpenRouter client; persist proposal.
POST /api/proposals/:id/decision	Record explicit approve/reject decision.
POST /api/findings/:id/regression	Run only preconfigured safe regression tests and persist results.
GET /api/runs/:id/report	Return stored run summary as JSON or Markdown.

8. Frontend integration

• Keep the existing frontend and call same-origin /api/* endpoints using fetch().

• Remove hard-coded demo-only state only when the equivalent backend endpoint is working.

• Show loading, empty, success and error states for database and AI requests.

• Render persisted run history after page refresh; do not rely only on browser memory.

• Show agent progress based on actual backend status. Label any seeded/sample content DEMO.

• Never put OPENROUTER_API_KEY or DATABASE_URL into HTML, JS bundles, localStorage or frontend environment variables.

9. Security and reliability requirements

• Create .env.example with placeholder values only; add .env, .env.* (except .env.example), local database files and logs to .gitignore.

• Validate all incoming request bodies and enforce request size limits, timeouts and rate limits.

• Redact Authorization headers, API keys, database URLs, tokens and secrets from logs and error responses.

• Use parameterized SQL queries or a safe query builder/ORM; never concatenate user input into SQL.

• Set OpenRouter request timeout, bounded retries for transient errors, and handle 401, 402/429, 5xx and malformed responses safely.

• Limit prompt size and model output tokens; pass only necessary source snippets and sanitized scanner evidence.

• Treat scanner output, repository files and model output as untrusted input.

• Keep scope authorization server-side. Use only local intentionally vulnerable training targets unless explicitly authorized.

• Human approval is required before any proposed patch is applied. No model-generated shell execution, automatic commit, push or merge.

10. Implementation phases and acceptance tests

Phase	Work	Acceptance test
1. Audit + config	Inspect actual repo; repair missing backend/routes if needed; add .env.example and .gitignore.	Server starts; health endpoint responds; no secrets in Git.
2. PostgreSQL	Add DB driver, connection pool, migrations and repositories.	Create a run, restart server, retrieve same run from DB.
3. OpenRouter	Add backend LLM client with configurable model and robust errors.	Backend test request succeeds; key is never sent to browser or logged.

Phase	Work	Acceptance test
4. Agent services	Connect agent-specific prompts and structured outputs to common LLM client.	Each agent can be invoked through backend and event is persisted.
5. UI wiring	Connect current UI to backend routes; show persisted history and actual status.	Reload page; run history and statuses remain available.
6. Verification	Add tests, error handling, usage tracking and security review.	Tests cover DB failure, invalid key, rate limit, model error and approval gate.

11. Antigravity master prompt

```
You are the senior backend engineer integrating PostgreSQL and OpenRouter into the EXISTING AEGIS-AGENT repository.

READ THIS SPECIFICATION FIRST. The repository currently describes itself as a frontend prototype. Inspect the actual checked-out project before making assumptions. package.json currently identifies a minimal Node.js server with no declared npm dependencies; server.js uses native http and imports ./backend/routes. Verify whether that backend path exists. Do not assume React/Vite/Express or rewrite the project.

OBJECTIVES
1. Connect the existing frontend to a persistent PostgreSQL database.
2. Integrate the team's existing OpenRouter API key through a server-only backend client.
3. Connect the Red, Sentinel, Fix and Auditor agent services to the backend AI client with separate responsibilities.
4. Persist scan runs, findings, evidence metadata, agent events, fix proposals, approvals, regression results and model usage.
5. Preserve the existing frontend design, navigation, workflow and safe scope restrictions.

REPOSITORY AUDIT FIRST
- Inspect the entire project tree, all package scripts, server.js, backend routes, frontend JS, CSS and README.
- Report missing imports, especially ./backend/routes, and identify current simulated/demo behavior.
- Propose a concise plan, then implement in phases. Do not stop after writing a plan.

ENVIRONMENT AND SECRETS
- Create .env.example containing PORT, DATABASE_URL, OPENROUTER_API_KEY, OPENROUTER_BASE_URL=https://openrouter.ai/api/v1, OPENROUTER_MODEL and APP_URL placeholders.
- Add dotenv or an equivalent safe environment loader if required by the chosen runtime.
- Ensure .env and local secrets are ignored by Git. Never ask the user to paste secrets into chat or commit them.
- OPENROUTER_API_KEY and DATABASE_URL must only be read by backend code. Never expose them to frontend JavaScript, browser storage, API responses, logs or screenshots.
- Use the OpenRouter chat completions endpoint through one reusable backend module. Read model ID from environment/config; do not hardcode an unverified model. Add timeout, bounded retry, token limits, safe error handling and redacted logs.

DATABASE
- Use PostgreSQL. Prefer the existing team's chosen provider if configured; otherwise document Supabase PostgreSQL setup as the hosted option and local PostgreSQL as an alternative.
- Add a connection pool, migration scripts, parameterized queries and repository/data-access modules.
- Create tables: scan_runs, findings, evidence, agent_events, remediation_proposals, approval_records, regression_results and model_usage. Add users only if authentication is being implemented.
- Add foreign keys, indexes and timestamps. Do not store secrets or unredacted credentials in the database.
- Add a health check that verifies service availability without returning credentials.

BACKEND API
Implement/repair routes for:
GET /api/health
GET/POST /api/runs
GET /api/runs/:id
GET /api/findings/:id
POST /api/findings/:id/propose-fix
POST /api/proposals/:id/decision
POST /api/findings/:id/regression
GET /api/runs/:id/report
Maintain compatibility with existing API behavior where possible and document any changes.

AGENT INTEGRATION
- Use one common OpenRouter client module, called only from backend agent services.
- Red Agent: use approved scanner/tool output; LLM may help analyze and prioritize. Do not let the LLM launch arbitrary attacks.
```

```
- Sentinel: assess supplied evidence and return structured reasoning; final verification status must be
evidence-backed, not model-only.
- Fix Agent: generate a proposed patch/diff and rationale from minimal relevant source context. Do not
apply automatically.
- Auditor: explain actual test output; pass/fail must come from real test process results, not model
opinion.
- Store agent events and model usage metadata. Avoid persisting prompts containing secrets or
unnecessary source code.
- Validate model outputs against schemas and treat them as untrusted.

FRONTEND
- Keep current UI and call same-origin backend endpoints.
- Replace mock state only as each backend feature becomes functional.
- Display real loading/error/empty states. Persist run history across page reloads.
- Clearly label demo/seeded records and never present simulated findings as real scans.

SECURITY
- Use parameterized SQL, input validation, request size limits and rate limiting.
- Redact keys, passwords, tokens and Authorization headers from logs.
- Keep scope allowlisting server-side; default to local intentionally vulnerable targets.
- No arbitrary public scanning, destructive probes, credential misuse or unrestricted network access.
- Never execute shell commands from model output.
- Require explicit human approval before patch application. No automatic commit, push or merge.

DELIVERY
- Implement one phase at a time and run tests after each.
- Test missing/invalid OpenRouter key, provider rate limits, model errors, database unavailable,
migrations, persistence after restart, malformed requests and approval enforcement.
- At completion report files changed, dependencies added, setup commands, required environment
variables, migrations, test results, verified functionality and limitations.
- Do not claim integration is complete until a real backend request and database read/write have been
successfully tested.
```

12. Setup checklist for the developers

• Create PostgreSQL database and copy its server-side connection URL.

• Use the already-created OpenRouter key locally in .env. If it has been exposed in Git, rotate it immediately.

• Copy .env.example to .env and fill in values locally; do not upload .env to Antigravity or GitHub.

• Install dependencies, run migrations and start the backend using the documented project commands.

• Verify /api/health, then create and retrieve a scan run from PostgreSQL.

• Test one OpenRouter request from the backend, then invoke each agent and inspect persisted events.

• Reload the frontend and confirm history is loaded from the database.

Repository reviewed: https://github.com/KulakeerthiKK/AfterHours (public main branch), README, package.json, server.js and root file listing as accessible on 30 October 2026. The README explicitly calls the application a frontend prototype with no real scans or code changes; the package manifest has no dependencies; server.js imports ./backend/routes. The checked-out tree must be verified by Antigravity because the public root listing does not show backend/.

Official integration references: OpenRouter API documentation: https://openrouter.ai/docs/api-reference/overview ; Supabase PostgreSQL docs: https://supabase.com/docs/guides/database/overview